Drowsy Bot

Drowsy Bot is a Discord bot for running stage-style voice events, publishing scheduled events, controlling invite-link moderation, managing reaction roles, routing moderation and server logs, and handling basic moderation workflows.

It is built with Node.js and discord.js and stores its runtime configuration in JSON files on disk.

Feature Summary

- Multi-stage queue flow for voice events
- Intermission radio playback from a local MP3 file
- Hype session controls with audience buttons
- Optional recording delivery for the active speaker
- Public event lookup using Discord scheduled events
- Invite-link moderation with allowlisting and cleanup tools
- Auto-updating stat channels and on-demand server stats summaries
- Reaction-role messages with multiple operating modes
- Logging for messages, invites, members, moderation, and server changes
- Moderation commands with persistent case history
- Sticky-role restoration for returning members

Requirements

- Node.js 20 or newer
- A Discord application with a bot user
- A Discord server where the bot can be invited and managed

Installation

1. Clone the repository.
2. Install dependencies.
3. Create a `.env` file.
4. Start the bot.

```
git clone https://github.com/ABZ-Digital-Group/drowsy_bot.git
cd drowsy_bot
npm install
node index.js
```

For a production process manager, `pm2` is a reasonable choice:

```
npm install
pm2 start index.js --name drowsy_bot
```

Environment Variables

The bot reads configuration from `.env` via `dotenv`.

Required values:

```
DISCORD_TOKEN=your_bot_token
CLIENT_ID=your_application_id
GUILD_ID=your_server_id
ALLOW_INVITE_PASSWORD=optional_dm_password
```

Variable Reference

- `DISCORD_TOKEN` : the bot token from the Discord developer portal
- `CLIENT_ID` : the Discord application ID used when registering slash commands
- `GUILD_ID` : the target guild where slash commands are registered
- `ALLOW_INVITE_PASSWORD` : optional DM password a user can send with `!allowinvite <password>` to self-authorize invite posting

Discord Permissions and Intents

The bot is configured with these gateway intents:

- Guilds
- GuildMessages
- MessageContent
- GuildVoiceStates
- GuildMembers
- DirectMessages
- GuildScheduledEvents
- GuildMessageReactions
- GuildModeration

The bot should also have practical server permissions for the features you enable, including:

- View Channels
- Send Messages
- Embed Links
- Read Message History
- Manage Messages
- Manage Roles
- Moderate Members
- Kick Members
- Ban Members
- Connect and Speak in voice channels

For queue hosting, role assignment, moderation, and cleanup actions, the bot must sit high enough in the role hierarchy.

Staff Access Model

Staff-only commands are available to:

- The guild owner
- Members with `Administrator`
- Members with `Manage Guild`
- Members with `Moderate Members`
- Members who hold one of these role names:

    - `Guards`
    - `Knights`
    - `Drowsy Defenders`
    - `God`

Project Structure

```
index.js
src/
  commands.js
  config.js
  helpers.js
  state.js
  features/
    community.js
    stage.js
recordings/
assets/
data/
```

Important Runtime Paths

- `assets/intermission.mp3` : background intermission audio for the stage radio flow
- `recordings/` : generated speaker recordings are written here temporarily
- `data/` : JSON persistence for guild settings and bot state

Persistent Data Files

The bot creates and maintains these files under `data/` :

- `guild-config.json` : log routing, ignore lists, and moderation helper config per guild
- `allowed-invite-users.json` : invite-link allowlist
- `member-stats.json` : persistent per-member message and voice analytics
- `reaction-roles.json` : stored reaction-role message definitions
- `modlog-cases.json` : moderation case history and next case number
- `sticky-roles.json` : roles to restore when members rejoin
- `temporary-reaction-roles.json` : pending temporary role expirations

Core Systems

Stage Queue System

The stage queue is designed for hosted performance or open-mic style events.

How It Works

1. A staff member runs `/start-queue` while connected to the target voice channel.
2. The bot posts a queue panel in the current text channel.
3. Users join the queue with the `Join Queue` button.
4. Staff move to the next performer with `/next` , or the current performer finishes with the `Done` button.
5. When no one is on stage, the bot can loop the intermission track.

Stage Buttons

- `Join Queue` : join the lineup if you are in the target voice channel
- `Leave` : leave the lineup
- `Done` : end your turn if you are the current speaker

Hype Session Buttons

- `Clap` : increases the hype meter
- `Fire` : increases the hype meter
- `Record Me` : starts a recording for the active speaker only

Stage Notes

- The queue panel is refreshed by deleting and re-posting the status message.
- The current speaker receives a dedicated hype message.
- Recordings are DM'd back to the active speaker if the output file is non-trivial.
- The intermission track is expected at `assets/intermission.mp3` .

Events System

The bot can publish active or upcoming Discord scheduled events.

Available entry points:

- `/events`
- `-events`

Behavior:

- Fetches scheduled events from the guild
- Filters to `Scheduled` and `Active`
- Sorts by start time
- Posts native Discord event URLs so clients render event cards naturally

Server Stats System

The bot includes a Statbot-style server stats feature for staff.

What It Can Do

- Show an on-demand summary of the server's current counts
- Show a Statbot-style member activity profile
- Rename configured voice or stage channels to display live metrics
- Refresh stat channels automatically when the server changes

Supported Metrics

- Total members
- Human members
- Bots
- Total non-category channels
- Text channels
- Voice channels
- Roles
- Members currently in voice

Member Analytics

The member stats profile is designed to feel closer to Statbot's user summary card.

It includes:

- account creation and server join dates
- message and voice rank within the server
- message totals for 1 day, 7 days, 14 days, and all-time tracked history
- voice totals for 1 day, 7 days, 14 days, and all-time tracked history
- top message channels
- top voice channels
- a rendered 14-day activity chart image
- current Discord presence activities when available

Message alias:

- `s?u`
- `s?u @member`
- `s?u 123456789012345678`

Important note:

- member analytics only include data collected after this feature is deployed
- existing historical Discord activity is not backfilled from before the bot tracked it

Auto Refresh Triggers

Configured stats channels are refreshed when:

- The bot starts
- A member joins or leaves
- A voice state changes
- A channel is created or deleted
- A role is created or deleted

Server Stats Commands

- `/server-stats show`
- `/server-stats user`
- `/server-stats channel`
- `/server-stats remove`
- `/server-stats refresh`
- `/server-stats list`

Invite Moderation

Invite links are detected using a regex that covers both Discord invite links and Discord event links.

Invite Rules

- Invite links are logged if invite logging is configured.
- Unauthorized invite links are deleted automatically.
- A short warning message is posted in-channel and removed after a few seconds.
- Exempt users are determined by the guild owner or the invite allowlist.

Invite Access Methods

- `/allow-invites target:<user>`
- `/revoke-invites target:<user>`
- DM command: `!allowinvite <password>`

Invite Cleanup

- `/purge-invites`
- Optional `messages_per_channel` scan limit
- Skips channels where the bot lacks view, history, or message-management permissions

Reaction Roles

Reaction roles are stored in persistent config and applied from message reactions.

Supported Modes

- `unique` : only one mapped role set from that message at a time
- `verify` : keep the role even if the reaction is removed
- `reversed` : flip add/remove behavior
- `binding` : remove the reaction if the linked role disappears from the member
- `temporary_minutes` : auto-remove granted roles after the configured duration
- `self_destruct_minutes` : delete the reaction-role message after the configured duration

Access Controls

Each reaction-role message can define:

- A whitelist of required roles
- A blacklist of blocked roles

If a member does not satisfy the access rules, their reaction is removed and no role is granted.

Reaction-Role Workflow

1. Use `/reaction-role post` to create the base embed.
2. Use `/reaction-role map` to attach emoji-to-role mappings.
3. Use `/reaction-role access` to define whitelist or blacklist rules.
4. Use `/reaction-role list` to inspect stored config.
5. Use `/reaction-role unmap` to remove mappings.

Reaction-Role Notes

- A single emoji can map to multiple roles.
- Role assignments only work if the bot can manage those roles.
- Temporary role removal tasks are restored after bot restart.
- Self-destruct timers are restored after bot restart.

Logging System

The bot supports separate channels for different log categories.

Log Categories

- Message logs
- Invite logs

- Member logs
- Server logs
- Moderation logs
- Drama channel output
- Highlight channel output

Ignore Rules

Logging can ignore:

- Specific channels
- Specific members
- Specific message prefixes

Logged Events

Message-related:

- Message delete
- Bulk delete
- Message edit

Member-related:

- Join
- Leave
- Member update
- Ban
- Unban

Server-related:

- Channel create
- Channel delete
- Channel rename
- Role create
- Role delete
- Role rename
- Emoji create
- Emoji delete
- Emoji rename

Invite-related:

- Seen invite links
- Unauthorized invite deletions

Moderation System

Moderation actions create persistent case records and can optionally emit to both a mod log channel and a drama channel.

Safety Checks

Before acting, the bot blocks:

- Self-targeting

- Targeting the guild owner
- Targeting members with equal or higher role position than the moderator
- Actions the bot cannot perform due to hierarchy or missing permissions

Stored Moderation Data

Each case stores:

- Case number
- Action
- Target
- Moderator
- Reason
- Timestamp

Sticky Roles

When configured, sticky roles are remembered when a member leaves and restored when they return.

The muted role is treated as part of the sticky-role preservation set if configured.

Command Reference

Public Commands

`/events`

Posts links for active or upcoming scheduled server events.

`-events`

Message-based shortcut for the same event lookup.

Invite Commands

`/allow-invites`

Allows a user or bot to post Discord invite links.

Arguments:

- `target` required

`/revoke-invites`

Removes invite-posting permission from a user or bot.

Arguments:

- `target` required

`/purge-invites`

Scans text channels and deletes unauthorized invite links.

Arguments:

- `messages_per_channel` optional

Server Stats Commands

`/server-stats user`

Shows activity stats for a member. If no member is provided, it shows your own tracked profile.

Arguments:

- `member` optional

Message alias:

- `s?u`
- `s?u @member`

`/server-stats show`

Shows the current server stats summary in an embed.

`/server-stats channel`

Assigns one metric to a voice or stage channel that the bot will keep renamed.

Arguments:

- `metric` required
- `channel` required

Metrics:

- `members`
- `humans`
- `bots`
- `channels`
- `text_channels`
- `voice_channels`
- `roles`
- `in_voice`

`/server-stats remove`

Removes a stat-channel binding.

Arguments:

- `metric` required

`/server-stats refresh`

Forces an immediate refresh of all configured stat channels.

`/server-stats list`

Shows the current stat-channel bindings.

Stage Commands

`/start-queue`

Starts the stage queue using the staff member's current voice channel.

`/stop-queue`

Stops the queue and destroys the voice connection.

`/next`

Moves to the next speaker.

`/radio`

Toggles intermission playback.

Reaction-Role Commands

`/reaction-role post`

Creates a reaction-role message.

Arguments:

- `channel` required
- `title` required
- `description` required
- `unique` optional
- `verify` optional
- `reversed` optional
- `binding` optional
- `temporary_minutes` optional
- `self_destruct_minutes` optional

`/reaction-role map`

Maps an emoji to a role on an existing reaction-role message.

Arguments:

- `channel` required
- `message_id` required
- `emoji` required
- `role` required

`/reaction-role unmap`

Removes a role from an emoji mapping.

Arguments:

- `channel` required
- `message_id` required

- `emoji` required
- `role` required

`/reaction-role access`

Updates whitelist or blacklist rules.

Arguments:

- `channel` required
- `message_id` required
- `list` required
- `action` required
- `role` required

`/reaction-role list`

Shows the stored config for a reaction-role message.

Arguments:

- `channel` required
- `message_id` required

Logging Commands

`/logging channel`

Assigns a channel for a logging category.

Arguments:

- `category` required
- `channel` required

Categories:

- `message`
- `invite`
- `member`
- `server`
- `mod`
- `drama`
- `highlight`

`/logging ignore-channel`

Adds or removes a channel from logging ignore rules.

Arguments:

- `action` required
- `channel` required

`/logging ignore-member`

Adds or removes a member from logging ignore rules.

Arguments:

- `action` required
- `member` required

`/logging ignore-prefix`

Adds or removes a message prefix from logging ignore rules.

Arguments:

- `action` required
- `prefix` required

`/logging status`

Shows the current logging configuration.

Moderation Commands

`/moderation warn`

Creates a warning case.

Arguments:

- `member` required
- `reason` required

`/moderation timeout`

Times out a member.

Arguments:

- `member` required
- `minutes` required
- `reason` required

`/moderation untimeout`

Removes an active timeout.

Arguments:

- `member` required
- `reason` optional

`/moderation kick`

Kicks a member.

Arguments:

- `member` required
- `reason` required

`/moderation ban`

Bans a member.

Arguments:

- `member` required
- `reason` required
- `delete_days` optional

`/moderation unban`

Unbans a user by raw user ID.

Arguments:

- `user_id` required
- `reason` optional

`/moderation purge`

Bulk deletes recent messages, optionally filtered to one member.

Arguments:

- `amount` required
- `member` optional
- `reason` optional

`/moderation history`

Shows the recent stored case history for a member.

Arguments:

- `member` required

`/moderation bulk-role`

Adds or removes one role for every member who currently has another role.

Arguments:

- `action` required
- `target_role` required
- `source_role` required
- `reason` optional

Moderation Config Commands

`/moderation-config muted-role`

Sets the muted role ID used by sticky-role handling.

Arguments:

- `role` required

`/moderation-config sticky-role`

Adds or removes a sticky role.

Arguments:

- `action` required
- `role` required

`/moderation-config show`

Shows the current moderation helper configuration.

First-Time Setup Checklist

1. Create the Discord application and bot.
2. Enable the intents used by the bot in the Discord developer portal.
3. Invite the bot to the server with the required permissions.
4. Add the `.env` file with valid IDs and token.
5. Run `npm install`.
6. Add `assets/intermission.mp3` if you want queue radio playback.
7. Start the bot so slash commands register for the configured guild.
8. Configure log channels with `/logging channel`.
9. Configure moderation helpers with `/moderation-config` if needed.
10. Configure `/server-stats channel` if you want Statbot-style counter channels.
11. Create reaction-role posts if you want self-serve role assignment.

Deployment Notes

Updating the Bot

Typical update flow:

```
git pull origin main
npm install
pm2 restart drowsy_bot
```

If you are not using `pm2`:

```
git pull origin main
npm install
node index.js
```

Missing Dependency Error

If you see an error like `Cannot find module 'dotenv'`, dependencies are not installed on that machine.

Fix:

```
npm install
```

Pull Conflicts Around `node_modules`

If `git pull` fails because files under `node_modules` would be overwritten, clean the working tree and reinstall dependencies:

```
rm -rf node_modules
git checkout -- package.json package-lock.json node_modules/.package-lock.json
git pull origin main
npm install
```

Troubleshooting

Slash Commands Do Not Appear

Check:

- `CLIENT_ID` is correct
- `GUILD_ID` is correct
- The bot can log in successfully
- The configured guild is the server you are testing in

Slash Command Registration Fails

If Discord reports `Invalid Form Body`, the command schema is invalid. Review recent changes to src/commands.js, especially required and optional option order.

Reaction Roles Do Not Apply

Check:

- The bot role is above the target roles
- The message ID and channel ID are correct
- The emoji mapping exists
- The member passes whitelist and blacklist checks

Queue Radio Does Not Play

Check:

- `assets/intermission.mp3` exists
- The bot can connect and speak in the voice channel
- The runtime has installed voice dependencies

Moderation Actions Fail

Check:

- The moderator is not targeting themselves
- The target is not the guild owner
- The moderator outranks the target
- The bot outranks the target
- The bot has the relevant moderation permission

Architecture Notes

The runtime is modularized into:

- index.js: bootstrap and event binding
- src/config.js: constants, paths, and environment access
- src/state.js: JSON-backed persistence and in-memory runtime state

- src/helpers.js: shared helper utilities
- src/commands.js: slash-command schema generation
- src/features/stage.js: queue, hype, radio, and recording flow
- src/features/community.js: events, invite moderation, reaction roles, logging, moderation, and event handlers

Limitations and Notes

- Slash commands are registered to a single guild, not globally.
- Recordings are delivered by DM and rely on voice receiving and FFmpeg behavior from the installed dependency stack.
- Reaction-role state and moderation history are file-backed, not database-backed.
- Logging only occurs for categories that have configured channels.
- Invite moderation exempts the guild owner and allowlisted users.

License

ISC